Los Alamos Chess Platform

L4 (Expert) Bot Integration Guide

Project Team: Byron, Ethan, Natasha, Arno, Elizabeth

26th September 2025

Contents

1 L4 Bot Integration Guide **1**

1.1 Overview 1

1.2 Setup Instructions 2

1.3 Backend Integration (Ethan – WebSocket) 4

1.4 Frontend Integration (Natasha – UI) 6

1.5 Database Considerations (Arno) 7

1.6 Testing 8

1.7 Deployment 9

1.8 Common Issues & Solutions 9

1.9 Contacts 10

Chapter 1

L4 Bot Integration Guide

1.1 Overview

This chapter explains how to integrate the new **L4** difficulty level into each subsystem of the Los Alamos Chess Platform. L4 introduces an expert-level opponent powered by **Fairy-Stockfish**, a variant-capable chess engine. While L0–L3 rely on custom algorithms, L4 delegates move selection to an external engine process with configurable strength, exposed through the same high-level interface used by the existing bot abstraction.

What is L4?

L4 is an expert opponent layer that consumes a legal position (FEN6x6), searches using Fairy-Stockfish, and returns a legal move with an evaluation. Its playing strength is driven by an *ELO-like* configuration that you can set per game. The integration preserves the existing bot contract, so all downstream consumers (frontend, WebSocket, persistence) continue to operate with minimal changes.

Key Features

L4 provides a configurable playing strength (nominal ELO 1000–3000), significantly outperforms L3, and conforms to the same request/response shape as L0–L3 so that existing pipelines and tests remain valid.

Difficulty Levels Summary

Level	Type	Strength	ELO Equivalent
L0	Random moves	Beginner	~ 400
L1	Greedy (1-ply)	Novice	~ 800
L2	Minimax (depth 2)	Intermediate	~ 1200
L3	Enhanced (depth 3)	Advanced	~ 1500
L4	**Fairy-Stockfish**	**Expert (configurable)**	**1000–3000**

1.2 Setup Instructions

All team members should install the Fairy-Stockfish binary locally and make it available to the backend.

Download Fairy-Stockfish

Obtain a release for your platform from the official repository:

- **Windows:** `fairy-stockfish-largeboard_x86-64-bmi2.exe`
- **Mac (Intel):** `fairy-stockfish-largeboard_x86-64-modern`
- **Mac (Apple):** `fairy-stockfish-largeboard_arm64`
- **Linux:** `fairy-stockfish-largeboard_x86-64-modern`

Install into Project

```
# Rename the file:
# Windows: fairy-stockfish.exe
# Mac/Linux: fairy-stockfish

# Place it under:
# backend/bin/

# Mac/Linux make it executable:
chmod +x backend/bin/fairy-stockfish
```

Listing 1.1: Place and prepare the engine binary

Verify Installation

```
npm run test:byron:fairy-stockfish
# Expected: " ALL TESTS PASSED"
```

Listing 1.2: Backend engine verification

Pull the Latest Code

```
git pull origin feature/backend-L4-Stockfish
# OR after merge:
git pull origin main
```

Listing 1.3: Sync the integration branch or main

If `node-uci` is Missing (Install Commands)

If `node-uci` is not present after running `npm install` (for example, the dependency was added later or your lockfile is stale), install it explicitly using one of the following commands for your package manager.

Install with npm (recommended).

```
npm install node-uci --save
```

Install node-uci with npm and persist to package.json

Alternative: Yarn.

```
yarn add node-uci
```

Install node-uci with Yarn

After installing, teammates only need to run the standard `npm install` on future checkouts; `node-uci` will be pulled automatically from `package.json`.

Quick verification (optional).

```
# Shows installed version if present:
node -p "require('node-uci/package.json').version"
# or
npm ls node-uci
```

Confirm node-uci is resolvable

Engine Binary Location (Required Path)

The Fairy-Stockfish binary **must** be placed at the following path inside the repository:

Windows (required path): `backend/bin/fairy-stockfish.exe`

macOS/Linux (required path): `backend/bin/fairy-stockfish`

This is the default location the backend expects. You can override it with `FAIRY_STOCKFISH_PATH`, but the standard path above is strongly recommended for team consistency.

Repository tree (expected):

```
backend/
  bin/
    fairy-stockfish.exe # Windows
    fairy-stockfish # macOS/Linux (executable)
```

Project tree showing required binary path

Windows: place and verify (PowerShell).

```
# From the repo root:
Rename-Item .\Downloads\fairy-stockfish-largeboard_x86-64-bmi2.exe fairy-stockfish.exe
New-Item -ItemType Directory -Force -Path .\backend\bin | Out-Null
Move-Item -Force .\fairy-stockfish.exe .\backend\bin\fairy-stockfish.exe

# Verify the exact path:
Test-Path .\backend\bin\fairy-stockfish.exe
```

Windows PowerShell commands

macOS/Linux: place, chmod, and verify (shell).

```
# From the repo root:
mv ~/Downloads/fairy-stockfish-largeboard_* ./backend/bin/fairy-stockfish
chmod +x ./backend/bin/fairy-stockfish

```

```
# Verify the exact path and permissions:
ls -l ./backend/bin/fairy-stockfish
```

macOS/Linux commands

Optional: override with environment variable.

```
# Only if you do NOT use the standard path:
# Windows (PowerShell):
$env:FAIRY_STOCKFISH_PATH="C:\tools\fairy-stockfish\fairy-stockfish.exe"

# macOS/Linux (shell):
export FAIRY_STOCKFISH_PATH="/opt/fairy-stockfish/fairy-stockfish"
```

Custom engine path override

Programmatic check (Node).

```
const fs = require('fs');
const path = process.env.FAIRY_STOCKFISH_PATH || 'backend/bin/fairy-stockfish' + (
    process.platform === 'win32' ? '.exe' : '');
if (!fs.existsSync(path)) {
  console.error('Engine not found at:', path);
  process.exit(1);
}
console.log('Engine found at:', path);
```

Sanity check that the binary exists at the required path

1.3 Backend Integration (Ethan – WebSocket)

The backend networking layer must forward the `elo` parameter for L4 and ensure the AI process is cleaned up on shutdown.

Bot Move Request Handler

```
// backend/networking/websocket-handlers.js

socket.on('botMoveRequest', async (data) => {
  const { fen, level, elo } = data; // Note: 'elo' is new for L4
  try {
    const botResponse = await aiBot.generateMove({
      fen,
      level,
      elo: elo || 2000, // default strength
      msCap: 3000 // time cap for move computation
    });

    if (botResponse.ok) {
      socket.emit('botMove', {
        move: botResponse.move,
        evaluation: botResponse.evaluation,
        ...(botResponse.elo && { elo: botResponse.elo })
      });
    } else {
```

```
      socket.emit('botError', {
        error: botResponse.error,
        details: botResponse.details
      });
    }
  } catch (error) {
    console.error('Bot move error:', error);
    socket.emit('botError', { error: 'INTERNAL_ERROR' });
  }
});
```

Listing 1.4: Handle L4 requests and propagate ELO

Server Shutdown Handler

```
// backend/networking/server.js
const aiBot = new AIBot();

process.on('SIGTERM', async () => {
  console.log('Shutting down gracefully...');
  await aiBot.cleanup(); // closes Fairy-Stockfish process
  server.close();
  process.exit(0);
});

process.on('SIGINT', async () => {
  console.log('Shutting down gracefully...');
  await aiBot.cleanup();
  server.close();
  process.exit(0);
});
```

Listing 1.5: Close Fairy-Stockfish on shutdown

Game Initialisation

```
const validLevels = ['L0', 'L1', 'L2', 'L3', 'L4'];
if (!validLevels.includes(gameData.botLevel)) {
  return { error: 'Invalid bot level' };
}

if (gameData.botLevel === 'L4') {
  const elo = gameData.elo || 2000;
  if (elo < 1000 || elo > 3000) {
    return { error: 'Invalid ELO rating (must be 1000-3000)' };
  }
}
```

Listing 1.6: Validate level and ELO at game creation

New WebSocket Events

Event	Direction	Data	Description
`botMoveRequest`	Client → Server	`{ fen, level, elo?}`	Requests a bot move; `elo` is optional a only used for L4.
`botMove`	Server → Client	`{ move, evaluation, elo?}`	Returns the move and evaluation; `elo` included for L4.

1.4 Frontend Integration (Natasha – UI)

Expose L4 in the difficulty selector, show an ELO control when L4 is chosen, and send it with bot requests.

Difficulty Selector

```jsx
// components/DifficultySelector.jsx
const difficulties = [
  { value: 'L0', label: 'Beginner', description: 'Random moves', icon: '' },
  { value: 'L1', label: 'Novice', description: 'Basic strategy', icon: '' },
  { value: 'L2', label: 'Intermediate', description: 'Tactical play', icon: '' },
  { value: 'L3', label: 'Advanced', description: 'Strategic depth', icon: '' },
  { value: 'L4', label: 'Expert', description: 'Stockfish Engine', icon: '', hasElo:
      true }
];
```

Listing 1.7: Add L4 with ELO capability

ELO Selector (Only for L4)

```jsx
const [selectedLevel, setSelectedLevel] = useState('L1');
const [elo, setElo] = useState(2000);

const eloPresets = [
  { value: 1200, label: 'Club Player' },
  { value: 1500, label: 'Intermediate' },
  { value: 1800, label: 'Advanced' },
  { value: 2000, label: 'Expert' },
  { value: 2200, label: 'Master' },
  { value: 2500, label: 'Grandmaster' },
];

// ... render cards ...

{selectedLevel === 'L4' && (
  <div className="elo-selector">
    <h3>Select Opponent Strength</h3>
    <div className="elo-presets">
      {eloPresets.map(p => (
        <button key={p.value}
               onClick={() => setElo(p.value)}
               className={elo === p.value ? 'active' : ''}>
          {p.label} ({p.value})
        </button>
```

```
      ))}
    </div>

    <div className="elo-slider">
      <label>Custom ELO: {elo}</label>
      <input type="range" min="1000" max="3000" step="50"
             value={elo}
             onChange={(e) => setElo(parseInt(e.target.value))}/>
    </div>
  </div>
)}
```

Listing 1.8: Preset buttons and custom slider

Send ELO with Bot Requests

```
// services/websocket.js
function requestBotMove(fen, level, elo) {
  const request = { fen, level };
  if (level === 'L4') request.elo = elo || 2000;
  socket.emit('botMoveRequest', request);
}
```

Listing 1.9: Include ELO only for L4

Display ELO in Game UI (Optional)

```
// components/GameBoard.jsx
{opponent === 'bot' && botLevel === 'L4' && (
  <div className="bot-info">
    <span>Playing against Expert Bot</span>
    <span className="bot-elo">ELO: {botElo}</span>
  </div>
)}
```

Listing 1.10: Show opponent ELO during play

UI/UX Notes

Show the selected ELO prominently, provide preset buttons for speed, and indicate when the L4 reply is pending, as engine moves typically arrive within a couple of seconds.

1.5 Database Considerations (Arno)

Store L4 strength either as a dedicated column or in a metadata JSON field.

Schema Updates

```
ALTER TABLE games
ADD COLUMN bot_elo INTEGER;
```

```
-- Only used when opponent_type = 'bot' AND bot_level = 'L4'
```

Listing 1.11: Option 1: dedicated ELO column

```
UPDATE games
SET metadata = jsonb_set(
  COALESCE(metadata, '{}'::jsonb),
  '{bot_elo}',
  '1800'
)
WHERE game_id = :game_id;
```

Listing 1.12: Option 2: JSON metadata

Queries to Update

```
-- Create game vs bot
INSERT INTO games (player_id, opponent_type, bot_level, bot_elo, ...)
VALUES (:player_id, 'bot', :bot_level, :bot_elo, ...);

-- Fetch game details with display name
SELECT *,
CASE
  WHEN bot_level = 'L4' THEN CONCAT(bot_level, ' (', bot_elo, ')')
  ELSE bot_level
END AS bot_display_name
FROM games
WHERE game_id = :game_id;
```

Listing 1.13: Persist and present ELO for L4 games

Statistics

Track win rate against L4 by ELO band, average opponent ELO, and the highest ELO defeated to enable meaningful progression metrics.

1.6 Testing

Test each component individually, then run a full-stack integration pass.

Unit and Component Tests

```
npm run test:byron:l4-bot
```

Listing 1.14: AI bot test suite

```
socket.emit('botMoveRequest', {
  fen: 'rnqknr/pppppp/6/6/PPPPPP/RNQKNR w - - 0 1',
  level: 'L4',
  elo: 1800
});

```

```
socket.on('botMove', (data) => {
  console.log('Bot move:', data.move);
  console.log('ELO used:', data.elo); // Expect: 1800
});
```

Listing 1.15: WebSocket L4 request with ELO

On the frontend, verify that L4 appears in the selector, the ELO slider renders only for L4, and the ELO value is included in requests.

Integration Testing

Launch the full stack, start a game versus L4 at ELO 1500, confirm the bot produces legal moves, ensure the stored game includes ELO, and repeat at multiple strengths (1000, 2000, 3000) to validate the pipeline and performance.

1.7 Deployment

Environment Variables

```
# Custom engine path (if not using backend/bin default)
FAIRY_STOCKFISH_PATH=/path/to/fairy-stockfish.exe

# Feature flag for L4
ENABLE_L4=true
```

Listing 1.16: Optional L4 configuration

Production Checklist

Engine installed on server; all tests pass; database schema migrated if needed; WebSocket handlers updated; frontend L4 UI built; error handling for missing engine; graceful shutdown wired; performance monitored (L4 typically responds within a few seconds per move on standard hosts).

Docker Deployment

```
# In your Dockerfile:
RUN curl -L https://github.com/fairy-stockfish/Fairy-Stockfish/releases/download/vXX/
    fairy-stockfish-largeboard_x86-64-modern \
    -o /app/backend/bin/fairy-stockfish && \
    chmod +x /app/backend/bin/fairy-stockfish
```

Listing 1.17: Add Fairy-Stockfish in Docker image

1.8 Common Issues & Solutions

Engine not found. Ensure `backend/bin/fairy-stockfish.exe` (or `fairy-stockfish` on Unix) exists and is executable.

Bot appears to move randomly. Engine failed to initialise; check logs and the binary path.

L4 not visible in UI. Pull latest code and rebuild the frontend.

WebSocket errors on L4 requests. Confirm the `elo` parameter is accepted and forwarded server-side.

Game stats lack ELO. Apply the database schema update and adjust queries as shown.

1.9 Contacts

For engine behaviour, configuration, or integration issues related to L4, contact the AI Bot developer (Byron).

Last Updated: September 26, 2025 Version: 1.0